

Babu Banarasi Das University



Case Study On

(UPI Transaction Analysis using Hadoop Framework)

SUBMITTED TO:

MR. Vikas

SUBMITTED BY:

Group A

Batch: BCADS23

Members:

Junaid Khan (1240258209)

Apoorv Sharma (1240258116)

Jai Prakash (1240258)

Bipendra Kumar Pandey (1240258)

Project Description

This project focuses on analyzing UPI-based banking transactions using the Hadoop ecosystem. With the rapid growth of digital payments, understanding transaction patterns, identifying peak usage times, and analyzing transaction behavior has become important.

The dataset contains transaction details such as amount, transaction type, status, time, and fraud indicators. The goal of this project is to process large-scale transaction data and extract meaningful insights using distributed computing techniques.

Architecture

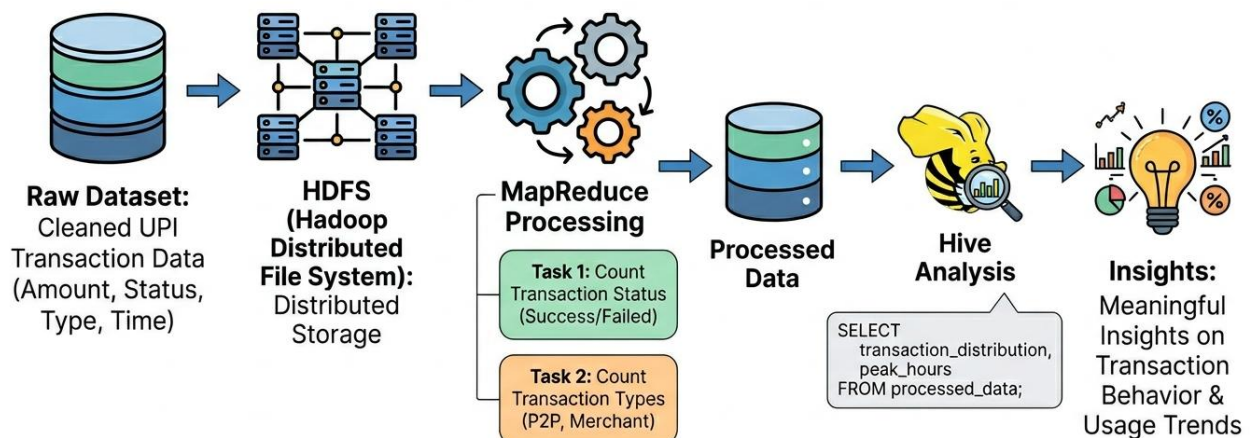
The project is built using the **Hadoop ecosystem**, which includes:

- HDFS (Hadoop Distributed File System): Used to store large transaction datasets in a distributed manner.
- MapReduce: Used to process and analyze data in parallel across nodes.
- Hive: Used to perform SQL-like queries for data analysis.
- Cloudera: Provides the environment to run Hadoop services.

This architecture ensures *scalability, fault tolerance, and efficient processing* of big data.

Data Flow

Raw Data → HDFS → MapReduce → Hive → Insights



Explanation

Data Upload to HDFS

The cleaned UPI transaction dataset, containing details like transaction amount, status, type, and time, is uploaded to HDFS for distributed storage.

Data Processing using MapReduce

MapReduce is applied to process the data by **counting transaction status** (success/failed) and **transaction types** (P2P, merchant).

Data Analysis using Hive

The dataset is then analyzed using Hive queries to identify patterns such as **transaction distribution and peak hours**.

Insights Generation

Finally, meaningful insights about transaction behavior and usage trends are derived from the results.

Tools And Technologies

Tools:

- Cloudera VM
- Hadoop Distributed File System (HDFS)
- MapReduce (Hadoop Streaming using Python)
- Hive (for data analysis)
- Linux Terminal
- ChatGPT (Generative AI tool)

Technologies:

- Python (for MapReduce using Hadoop Streaming)
- SQL (Hive Query Language)
- CSV dataset

Use of Generative AI in the Project

Generative AI tools such as ChatGPT were used during the development of this project to assist in various stages of implementation. These tools helped in simplifying complex tasks, improving code quality, and speeding up the development process.

Areas Where AI Was Used:

- **Code Generation:**
AI was used to generate MapReduce programs (mapper and reducer) using Python.
- **Debugging:**
AI helped in identifying and fixing errors related to Python version compatibility and Hadoop execution.
- **Query Writing:**
AI assisted in writing Hive queries for analyzing transaction data.

Benefits of Using AI

- **Faster Development:**
Reduced the time required to write and test code.
- **Error Reduction:**
Helped in debugging and improving code accuracy.
- **Better Logic Suggestions:**
Provided optimized approaches for data processing and analysis.

Project Implementation

Dataset Description:

The dataset used in this project is a UPI transaction dataset containing information about digital payment activities. It includes attributes such as transaction ID, merchant category, transaction amount, transaction status, transaction type, time details (hour and day), and fraud indicators. This structured dataset is used to analyze transaction patterns, usage trends, and overall payment behavior.

Overview of dataset:

transaction_id	merchant_category	amount_(INR)	transaction_status	transaction_type	hour_of_day	day_of_week	fraud_flag
TXN0000000001	Entertainment	868	SUCCESS	P2P	15	Tuesday	0
TXN0000000002	Grocery	1011	SUCCESS	P2M	6	Thursday	0
TXN0000000003	Grocery	477	SUCCESS	P2P	13	Tuesday	0
TXN0000000004	Fuel	2784	SUCCESS	P2P	10	Sunday	0
TXN0000000005	Shopping	990	SUCCESS	P2P	19	Tuesday	0

Implementation Steps:

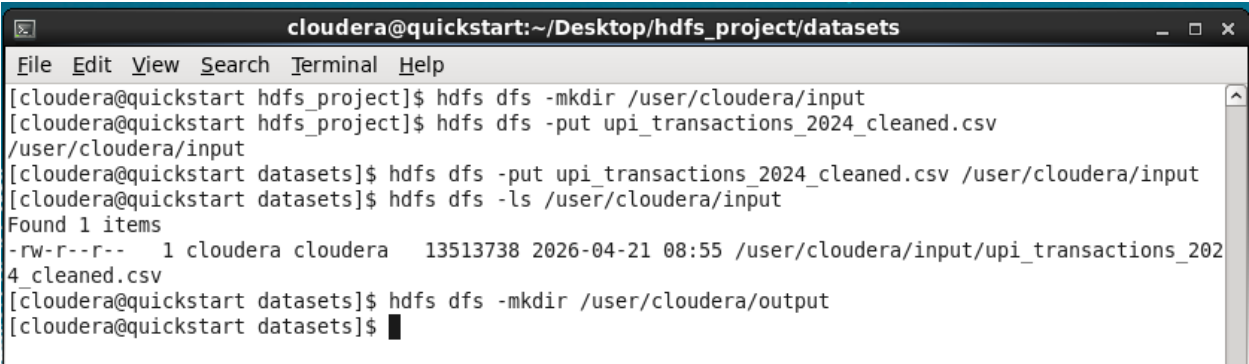
Step 1: HDFS Setup and Data Upload:

- Created input directory in HDFS:

```
hdfs dfs -mkdir -p /user/cloudera/input
```

- Uploaded dataset to HDFS:

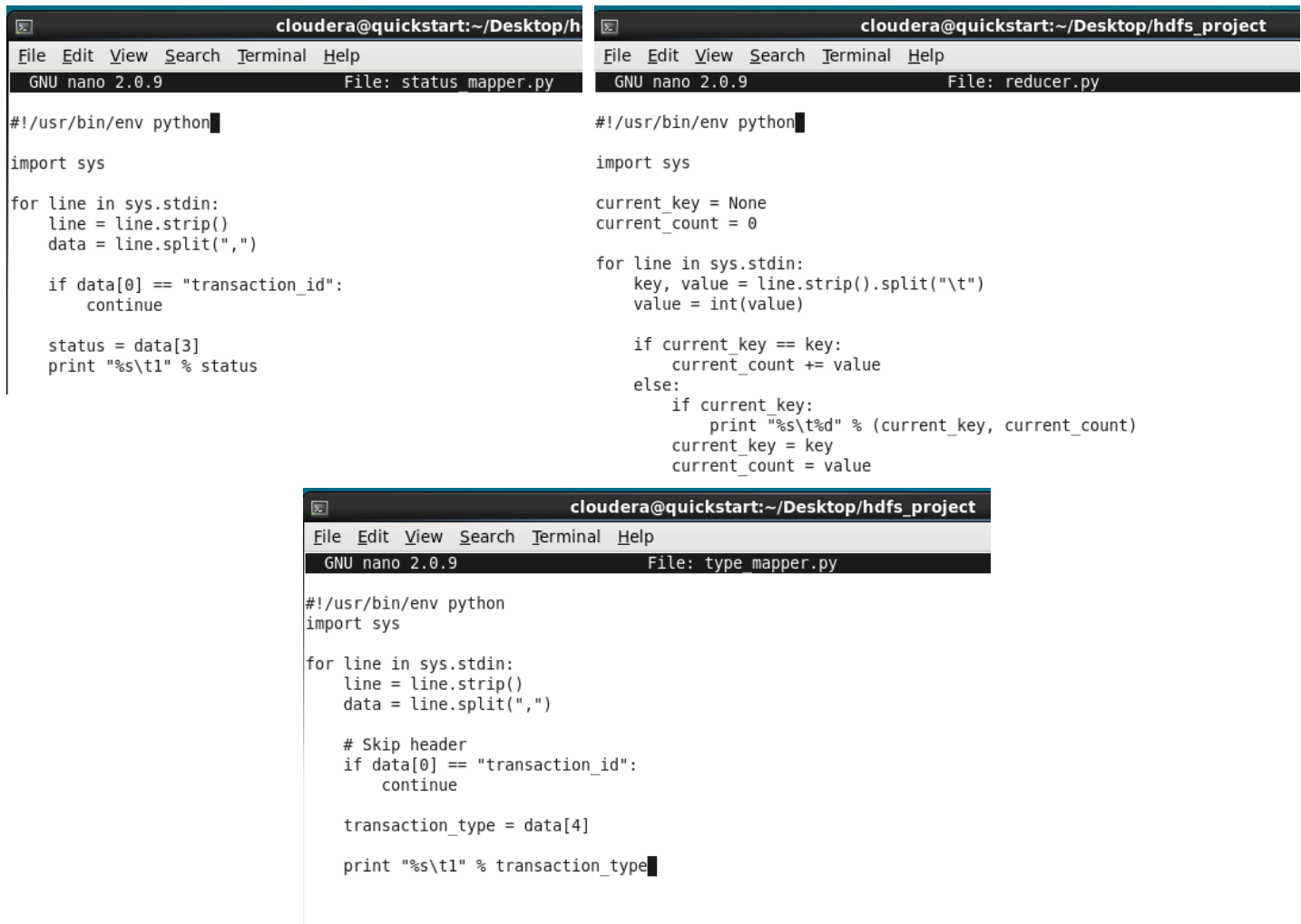
```
hdfs dfs -put upi_transactions_2024_cleaned.csv /user/cloudera/input
```



Step 3: MapReduce Implementation

- Developed Python-based mapper and reducer using Hadoop Streaming.
- **Task 1:** Count transaction status (success/failed)
- **Task 2:** Count transaction types (P2P, merchant)

The map reduce code is provided in the repo



```
cloudera@quickstart:~/Desktop/h  
File Edit View Search Terminal Help  
GNU nano 2.0.9 File: status_mapper.py  
#!/usr/bin/env python  
  
import sys  
  
for line in sys.stdin:  
    line = line.strip()  
    data = line.split(",")  
  
    if data[0] == "transaction_id":  
        continue  
  
    status = data[3]  
    print "%s\t1" % status
```

```
cloudera@quickstart:~/Desktop/hdfs_project  
File Edit View Search Terminal Help  
GNU nano 2.0.9 File: reducer.py  
#!/usr/bin/env python  
  
import sys  
  
current_key = None  
current_count = 0  
  
for line in sys.stdin:  
    key, value = line.strip().split("\t")  
    value = int(value)  
  
    if current_key == key:  
        current_count += value  
    else:  
        if current_key:  
            print "%s\t%d" % (current_key, current_count)  
            current_key = key  
            current_count = value
```

```
cloudera@quickstart:~/Desktop/hdfs_project  
File Edit View Search Terminal Help  
GNU nano 2.0.9 File: type_mapper.py  
#!/usr/bin/env python  
import sys  
  
for line in sys.stdin:  
    line = line.strip()  
    data = line.split(",")  
  
    # Skip header  
    if data[0] == "transaction_id":  
        continue  
  
    transaction_type = data[4]  
  
    print "%s\t1" % transaction_type
```

Image 1 mapper for status, image 2 reducer.py for both MapReduce operation, image 3 mapper for type

Performing task 1:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ nano status_mapper.py
[cloudera@quickstart hdfs_project]$ nano reducer.py
[cloudera@quickstart hdfs_project]$ nano type_mapper.py
[cloudera@quickstart hdfs_project]$ chmod +x status_mapper.py
[cloudera@quickstart hdfs_project]$ chmod +x reducer.py
[cloudera@quickstart hdfs_project]$ chmod +x type_mapper.py
[cloudera@quickstart hdfs_project]$
```

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -input /user/cloudera/input/upi_transactions_2024_cleaned.csv \
> -output /user/cloudera/output/status_result \
> -mapper status_mapper.py \
> -reducer reducer.py \
> -file status_mapper.py \
> -file reducer.py
```

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
Reduce input records=250000
Reduce output records=2
Spilled Records=500000
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=311
CPU time spent (ms)=20000
Physical memory (bytes) snapshot=725516288
Virtual memory (bytes) snapshot=4724469760
Total committed heap usage (bytes)=746061824
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=13515269
File Output Format Counters
Bytes Written=28
26/04/21 09:38:00 INFO streaming.StreamJob: Output directory: /user/cloudera/output/status_result
[cloudera@quickstart hdfs_project]$
```

Output of Task 1:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ hdfs dfs -cat /user/cloudera/output/status_result/part-000000
failed 12376
success 237624
[cloudera@quickstart hdfs_project]$
```

Performing task 2:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -input /user/cloudera/input/upi_transactions_2024_cleaned.csv \
> -output /user/cloudera/output/type_result \
> -mapper type_mapper.py \
> -reducer reducer.py \
> -file type_mapper.py \
> -file reducer.py
```

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
Reduce input records=250000
Reduce output records=4
Spilled Records=500000
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=169
CPU time spent (ms)=17680
Physical memory (bytes) snapshot=674201600
Virtual memory (bytes) snapshot=4728012800
Total committed heap usage (bytes)=618135552
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=13515269
File Output Format Counters
  Bytes Written=55
26/04/21 09:41:26 INFO streaming.StreamJob: Output directory: /user/cloudera/output/type_result
[cloudera@quickstart hdfs_project]$
```

Output of Task 2:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ hdfs dfs -cat /user/cloudera/output/type_result/part-00000
bill payment      37368
p2m               87660
p2p              112445
recharge          12527
[cloudera@quickstart hdfs_project]$
```


Step 4: Hive Implementation:

1. Created Hive table for the dataset:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> CREATE TABLE upi_transactions (
  >   transaction_id STRING,
  >   merchant_category STRING,
  >   amount INT,
  >   transaction_status STRING,
  >   transaction_type STRING,
  >   hour_of_day INT,
  >   day_of_week STRING,
  >   fraud_flag INT
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ',';
OK
Time taken: 22.403 seconds
hive>
```

2. Loaded data into Hive:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
hive> LOAD DATA INPATH '/user/cloudera/input/upi_transactions_2024_cleaned.csv'
  > INTO TABLE upi_transactions;
Loading data to table default.upi_transactions
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/upi_transactions/upi_transactions_2024_cleaned.csv': User does not belong to hive
Table default.upi_transactions_stats: [numFiles=1, totalSize=13513738]
OK
Time taken: 7.822 seconds
hive>
```

3. Verifying the loaded Data:

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
hive> SELECT * FROM upi_transactions LIMIT 10;
OK
transaction_id merchant_category amount transaction_status transaction_type N
ULL day_of_week NULL
TXN0000000001 entertainment 868 success p2p 15 tuesday 0
TXN0000000002 grocery 1011 success p2m 6 thursday 0
TXN0000000003 grocery 477 success p2p 13 tuesday 0
TXN0000000004 fuel 2784 success p2p 10 sunday 0
TXN0000000005 shopping 990 success p2p 19 tuesday 0
TXN0000000006 food 91 success p2p 22 monday 0
TXN0000000007 other 314 success p2p 10 thursday 0
TXN0000000008 utilities 264 success p2p 18 sunday 0
TXN0000000009 other 887 success p2p 9 thursday 0
Time taken: 5.959 seconds, Fetched: 10 row(s)
hive>
```

Step 5: Data Analysis using Hive:

1. Transaction Status Distribution:

```
SELECT transaction_status, COUNT(*)
FROM upi_transactions
GROUP BY transaction_status;
```

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
hive> SELECT transaction_status, COUNT(*)
> FROM upi_transactions
> GROUP BY transaction_status;
Query ID = cloudera_20260422032525_4505fcbf-0dc2-466a-966f-cdd470e93785
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
```

```
Total MapReduce CPU Time Spent: 1 minutes 4 seconds 560 m
OK
failed 12376
success 237624
transaction_status      1
Time taken: 375.747 seconds, Fetched: 3 row(s)
hive>
```

Insight:

The majority of transactions are **successful (237,624)** compared to failed transactions (12,376), indicating a highly reliable and efficient UPI payment system.

2. Peak Transaction Hours:

```
SELECT hour_of_day, COUNT(*) AS total
FROM upi_transactions
GROUP BY hour_of_day
ORDER BY total DESC;
```

```
cloudera@quickstart:~/Desktop/hdfs_p
File Edit View Search Terminal Help
hive> SELECT hour_of_day, COUNT(*) AS total
> FROM upi_transactions
> GROUP BY hour_of_day
> ORDER BY total DESC;
Query ID = cloudera_20260422034141_083cbcd0-3c98-44a1-b4e4-
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input
In order to change the average load for a reducer (in bytes)
```

```
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 38.14 sec HDFS Read:
ite: 618 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 49.78 sec HDFS Read:
200 SUCCESS
Total MapReduce CPU Time Spent: 1 minutes 27 seconds 920 msec
OK
19      21232
18      20064
20      18506
17      18340
```

Insight:

The highest number of transactions occurs around **7 PM (hour 19)**, followed by **6 PM and 8 PM**, indicating that evening is the peak usage time for UPI transactions.

3. Merchant Category Usage:

```
SELECT merchant_category, COUNT(*)
FROM upi_transactions
GROUP BY merchant_category;
```

```
cloudera@quickstart:~/Desktop/hdfs_project
File Edit View Search Terminal Help
hive> SELECT merchant_category, COUNT(*)
> FROM upi_transactions
> GROUP BY merchant_category;
Query ID = cloudera_20260422035454_8b3c99ed-be94-4e09-9ba1-d42ed5
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data s
In order to change the average load for a reducer (in bytes):
```

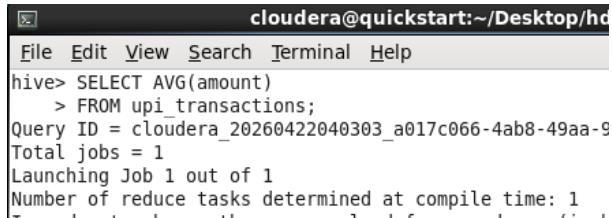
```
Total MapReduce CPU Time Spent: 39 seconds 100 msec
OK
education      7598
entertainment  20103
food           37464
fuel           25063
grocery        49966
```

Insight:

Transactions related to **fuel and utilities** are consistently high, suggesting that users rely on UPI for routine payments like fuel and bill payments.

4. Average Transaction Amount

```
SELECT AVG(amount)
FROM upi_transactions;
```



```
cloudera@quickstart:~/Desktop/hd
File Edit View Search Terminal Help
hive> SELECT AVG(amount)
> FROM upi_transactions;
Query ID = cloudera_20260422040303_a017c066-4ab8-49aa-9
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
Total MapReduce CPU Time Spent: 1 minutes 28 seconds 180 msec
OK
1311.756036
Time taken: 509.324 seconds, Fetched: 1 row(s)
hive>
```

Insight:

Average Amount → 1311.75 INR

The average transaction amount highlights UPI's role as a convenient medium for routine and mid-range financial transactions.

End of The Project

Thank You